

UNIVERSITI TUNKU ABDUL RAHMAN

ACADEMIC YEAR 2019/2020

SEPTEMBER EXAMINATION

UCCC2063 ALGORITHMS ANALYSIS

TUESDAY, 3 SEPTEMBER 2019

TIME : 9.00 AM – 11.00 AM (2 HOURS)

BACHELOR OF COMPUTER SCIENCE (HONS)
BACHELOR OF INFORMATION TECHNOLOGY (HONS)
COMPUTER ENGINEERING

Instruction to Candidates:

This question paper consists of THREE (3) questions in Section A and TWO (2) questions in Section B.

Answer **ALL** questions in **Section A** and **ONLY ONE (1)** question in **Section B**.
Each question carries 25 marks.

Should a candidate answer more than ONE (1) question in section B, marks will only be awarded for the FIRST ONE (1) question in that section in the order the candidate submits the answers.

Candidates are NOT allowed to use calculator.

Answer questions only in the answer booklet provided.

UCCC2063 ALGORITHMS ANALYSIS**SECTION A (Answer All Questions)**

Q1. [Asymptotic Notations]

(a) Express the following functions in terms of Big-Oh notations.

(i) $2n^{0.5} \log_3 n^{100} + 12n^{0.7}$ (1 mark)

(ii) $(70n)^{77} + n^{3333}(\log_2 n)^{123} + 0.001(2^n) + 2^{1.5n}$ (1 mark)

(iii) $2(n-1)! + n^5 + 9n^{700} + n(n!)$ (1 mark)

(iv) $60n^2 + 24n \log_2 n + 2019$ (1 mark)

(v) $\log_{30} n^{2019}$ (1 mark)

(vi) $33n \log_2 \log_2 \log_2 n + 2n \log_2 \log_2 n + 248n$ (1 mark)

(b) Using the definition of Big-Oh, prove $2000n^{10} = O(n^{10.01})$. (6 marks)
[Hint: $O(f(n)) = \{g(n) \mid \exists c, n_0 \bullet \forall n > n_0 \bullet g(n) \leq c f(n)\}$](c) Using the definition of Ω , prove $2000n^{10} \neq \Omega(n^{10.01})$. (6 marks)
[Hint: $\Omega(f(n)) = \{g(n) \mid \exists c, n_0 \bullet \forall n > n_0 \bullet g(n) \geq c f(n)\}$](d) Prove $\forall c \bullet \exists n_0 \bullet \forall n > n_0 \bullet 2^{6n} < c(n!)$. (7 marks)
[Hint: Note that c is defined by the universal quantifier.]

[Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS**Q2. [Solving Recurrence]**

- (a) Consider the recurrence $T(n)=3T(n/3)+n$.
- (i) Solve $T(n)$ by substitution (also called Mathematical Induction). Give your answer in Big-Oh notation. (4 marks)
 - (ii) Solve $T(n)$ by recursive method (also called expansion). Give your answer in Big-Theta notation. (4 marks)
- (b) Using master method to solve the following recurrences.
- (i) $T(n)=3T(n/3)+n \log n$. (4 marks)
 - (ii) $T(n)=3T(n/3)+n^{0.5} \log n$ (4 marks)
 - (iii) $T(n)=3T(n/3)+n^{1.5} \log n$. (4 marks)
- (c) Solving $T(n) = aT(n/b)+f(n)$ by case 3 of master method, we need to do the regularity checking, which is $\exists c < 1, n_0 > 0 \bullet \forall n > n_0 \bullet af(n/b) \leq cf(n)$. Prove that it is equivalent to use the Boolean expression $\exists c < 1, n_0 > 0 \bullet \forall n > n_0 \bullet af(n/b) < cf(n)$. (5 marks)
- [Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS**Q3. [Graph Algorithms]**

Consider an undirected graph $G=(V,E)$, where V is the set of vertices, E is the set of edges, $|V|=n$, and $|E|=e$. Answer the following questions.

- (a) Describe an adjacency list for storing G , and explain why $O(n+e)$ space is needed, and why both n and e are necessary in the Big-Oh notation. (5 marks)
- (b) Referring to the adjacency list in (a), give an algorithm for removing an edge, and an algorithm for adding an edge. (6 marks)
- (c) What are the time complexities for your algorithms in (b) in terms of n and e , whichever appropriate? (4 marks)
- (d) Suppose that the maximum degree over all nodes is d . What are the time complexities for your algorithms in (b) in terms of d ? Explain. (3 marks)
- (e) Give a space-efficient way for storing G in a file. (5 marks)
- (f) What is the space needed for storing G in an adjacency matrix? Explain. (2 marks)

[Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS**SECTION B (Choose Any One Question)**

Q4. [Dynamic Programming]

Consider the matrix chain multiplication problem. The input is a set of $n+1$ integers $\{p_0, p_1, \dots, p_n\}$, where the dimension of the matrix M_i is $p_{i-1} \times p_i$, and $i \in [1, n]$. The matrix chain multiplication problem is to find the order of multiplications for computing $M_1 \times M_2 \times \dots \times M_n$ such that the total cost will be minimized. The cost of computing AB is xyz , where matrices A and B have dimensions $x \times y$, and $y \times z$.

Referring to the cost of computing $M_i \times M_{i+1} \times \dots \times M_j$, where $i \leq j$, answer the following questions.

- (a) What is the cost when $j = i$? (2 marks)
- (b) What is the cost when $j = i + 2$? (4 marks)
- (c) Give a recursive formula for the minimum cost. (6 marks)
- (d) Based on the formula in (c), give a recursive algorithm for finding the minimum cost. (5 marks)
- (e) Based on the formula in (c), give a polynomial time algorithm for finding the minimum cost. (8 marks)

[Total : 25 marks]

UCCC2063 ALGORITHMS ANALYSIS

Q5. [Correctness proof, and Time Complexity Analysis]

Consider the following recursive selection sort algorithm:

```

1. Selection(A, p, r)
2.   If  $p \geq r$  then return;
3.   Find  $j \in [p, r]$  •  $A[j]$  stores the largest element;
4.   Swap  $A[j]$  and  $A[r]$ ;
5.   Call Selection(A, p, r-1)
6.   return;
```

The input is an array $A[p..r]$ which contains $r-p+1$ arbitrarily ordered elements. In particular, when **Selection(A, 1, N)** is called, the input is $A[1..N]$.

(a) You are given a statement:

Selection(A, 1, N) sorts the array $A[1..N]$ in ascending order.

Prove by induction on N that the statement is true for all $N \geq 1$.

[Hint: must contain base case, and inductive step.]

(10 marks)

(b) For a particular recursion call where $r-p+1=N$, answer the following questions using the Big-Oh notation, and in terms of N whenever appropriate.

(i) What is the time complexity of for a single execution of line 2?

(ii) What is the time complexity of a single execution of line 3?

(iii) What is the time complexity of a single execution of line 4?

(iv) What is the time complexity of a single execution of line 6?

(5 marks)

(c) Give the recurrence for the time complexity of **Selection(A, 1, N)**, and solve it.

(5 marks)

(d) Give a scenario of execution of **Selection(A, 1, N)** such that there exists an element which is swapped for $N-1$ times before the end of the algorithm.

(5 marks)

[Total : 25 marks]